

Jadavpur University



Computer Networks Lab Report

NAME : ANKIT SHAW

ROLL NO. : 002310501032

YEAR : BCSE UG – III (A1)

SESSION : 2023 – 27

ASSIGNMENT : 0 3

Implementation and Performance Analysis of CSMA Protocols

Abstract

This report details the design and implementation of a multi-threaded Python simulation to analyze and compare the performance of three fundamental Carrier Sense Multiple Access (CSMA) protocols: 1-Persistent, Non-Persistent, and P-Persistent. These Medium Access Control (MAC) protocols are essential for managing access to a shared communication channel. The simulation models multiple stations attempting to transmit data over a single shared channel, with mechanisms to sense the channel's state (idle or busy). The objective is to evaluate the protocols based on key performance metrics, namely throughput and average frame delay, under configurable network conditions such as the number of stations and simulation duration.

1 Introduction :

In networks where multiple devices share a single communication medium (e.g., wireless networks or early Ethernet), a mechanism is required to coordinate access and prevent data collisions. Medium Access Control (MAC) protocols serve this purpose. Carrier Sense Multiple Access (CSMA) is a family of probabilistic MAC protocols where each station "listens" to the channel (carrier sense) before attempting to transmit. This simple principle significantly reduces the probability of collisions compared to protocols like ALOHA.

This project implements a simulation environment in Python to model and contrast three primary variants of CSMA:

- 1-Persistent CSMA: An aggressive protocol where a station transmits as soon as the channel becomes idle.
- Non-Persistent CSMA: A more conservative protocol where a station backs off for a random time if the channel is busy.
- P-Persistent CSMA: A compromise between the two, which transmits with a probability 'p' when the channel becomes idle.

The simulation utilizes multi-threading to model the concurrent and independent behavior of each station, providing a practical platform to observe the inherent trade-offs between channel utilization, collision avoidance, and transmission delay in these protocols.

2. Project Objectives

The main objectives of this assignment are:

- To design and implement a multi-threaded simulation of a shared communication channel.

- To implement the logic for 1-Persistent, Non-Persistent, and P-Persistent CSMA protocols.
- To allow for user-configurable simulation parameters, including the number of stations, the choice of protocol, and the 'p' value for P-Persistent CSMA.
- To measure and calculate key performance metrics: system throughput and average frame delay.
- To analyze and compare the behavior of the protocols based on the simulation results.

3. System Design and Architecture

The simulation is built around two core classes: **Channel** and **Station**, leveraging Python's threading module to achieve concurrency.

3.1 Channel Class

The Channel class acts as a centralized, static entity representing the shared communication medium. It holds the global state and configuration for the simulation. Its key components are:

- **Shared State:** A boolean flag `isbusy` indicates the channel's current state. A `threading.Lock` is used to ensure that access to this shared flag is atomic, preventing race conditions between stations.
- **Simulation Parameters:** Contains constants like frame size, transmission time, and slot time, which govern the timing of the simulation.
- **Performance Metrics:** Global counters for successful transmissions and total delay are maintained. These are updated by stations after a successful transmission, protected by the shared lock.

3.2 Station Class

The Station class, a subclass of `threading.Thread`, models an individual network node that generates and attempts to transmit frames. Each station runs in its own thread, simulating independent and concurrent operation.

- **Initialization:** Each station is initialized with a unique ID and the specific CSMA method it must follow.
- **Main Loop:** The `run()` method contains the primary logic loop, where the station continuously attempts to transmit frames as long as the global simulation flag is active.
- **Transmission Logic:** The core of the station is its `acquiremedium()` method, which acts as a dispatcher to call the appropriate CSMA logic (1-persistent, non-persistent, or p-persistent) based on its configuration. Once the medium is acquired, it simulates sending a frame and then updates the global performance metrics.

3.3 Concurrency Model

Concurrency is central to this simulation. Each station operates independently. The critical section of the code is any access to the shared `Channel.isbusy` flag and the performance counters. A single mutex (`Channel.lock`) is used to protect these shared resources. A station must acquire the lock before checking or changing the channel's state, and also before updating the performance counters, ensuring data consistency.

4 Implementation of CSMA Protocols

The behavioral differences between the CSMA protocols are encapsulated within the medium acquisition logic of the `Station` class.

4.1 1-Persistent CSMA

This is the most aggressive strategy. If a station has a frame to send, it continuously senses the channel. If the channel is idle, it transmits immediately. If the channel is busy, it waits and continues to sense the channel, transmitting the moment it becomes free. This can lead to high rates of collision if two or more stations are waiting for the channel to become free, as they will both attempt to transmit at the same time.

4.2 Non-Persistent CSMA

This strategy is less aggressive and aims to reduce collisions. If the channel is idle, it transmits immediately. If the channel is busy, it does not continuously sense. Instead, it waits for a random amount of time (backs off) and then repeats the entire process. This reduces the likelihood of collision but can lead to lower channel utilization.

4.3 P-Persistent CSMA

This protocol is a compromise, primarily used in slotted channels. It continuously senses the channel until it becomes idle. Once the channel is idle, it transmits with a probability p . With probability $1-p$, it defers its transmission to the next time slot and repeats the process.

5. Code:

i. client.py :

```
import time
import random
import threading
import os
```

```
P_PERSISTENCE = 0.5
MAX_ATTEMPTS = 10
SLOT_TIME = 0.05
NUM_STATIONS = 4
FRAME_SIZE = 64
FRAMES_PER_STATION = 10
```

```

def generate_random_frames(num_frames,
frame_size=FRAME_SIZE):
    """Generate random binary data frames."""
    frames = [os.urandom(frame_size) for _ in
range(num_frames)]
    return frames

def run_station(station_id, frames_to_send):
    """one CSMA/CD station """
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    try:
        client_socket.connect(("localhost", 8080))
    except ConnectionRefusedError:
        print(f"Station {station_id} :Connection failed. Is
the server running?")
        return

    print(f"[Station {station_id}] Started with p =
{P_PERSISTENCE}")
    frames_sent_successfully = 0
    num_frames_to_send = len(frames_to_send)

    for i, frame_data in enumerate(frames_to_send):
        frame_num = i + 1
        current_attempts = 0

        print(f"\nStation {station_id} Sending Frame
{frame_num}/{num_frames_to_send} ")

        while current_attempts < MAX_ATTEMPTS:
            # 1. Carrier Sense
            client_socket.sendall("SENSE".encode())
            response = client_socket.recv(1024).decode()

            if response == "BUSY":
                print(f"[Station {station_id}] Medium
BUSY. Waiting...")
                time.sleep(SLOT_TIME)
                continue

            # 2. Channel IDLE
            if random.random() > P_PERSISTENCE:
                print(f"Station {station_id} p-check failed.
Wait 1 slot.")
                time.sleep(SLOT_TIME)
                continue

            # 3. Transmit frame
            print(f"Station {station_id} Transmitting frame
{frame_num}...")

```

```

        frame_start_time = time.time()
        client_socket.sendall(f"TRANSMIT | {frame_str
t_time}".encode())

    # 4. Wait for ACK or COLLISION
    result = client_socket.recv(1024).decode()

    if result == "SUCCESS":
        print(f"Station {station_id}:Frame
{frame_num} sent successfully!")
        frames_sent_successfully += 1
        break
    elif result == "COLLISION":
        current_attempts += 1
        print(f"Station {station_id} :Collision
detected (Attempt {current_attempts})")

        # Exponential Backoff
        backoff_limit = (2 ** current_attempts) - 1
        backoff_slots = random.randint(0,
backoff_limit)
        wait_time = backoff_slots * SLOT_TIME
        print(f"Station {station_id}:Backing
off{backoff_slots} slots ({wait_time:.2f}s)")
        time.sleep(wait_time)

    if current_attempts >= MAX_ATTEMPTS:
        print(f"Station {station_id}: Dropped Frame
{frame_num} after {MAX_ATTEMPTS} attempts.")

    print(f"\nStation {station_id}: Finished. Successful
frames:
{frames_sent_successfully} / {num_frames_to_send}")
    client_socket.close()

def main():
    # for all stations
    all_stations_frames =
[generate_random_frames(FRAMES_PER_STATION
) for _ in range(NUM_STATIONS)]

    # threads
    threads = []
    for i in range(NUM_STATIONS):
        t = threading.Thread(target=run_station, args=(i
+ 1, all_stations_frames[i]))
        t.start()
        threads.append(t)
        time.sleep(0.2) # small delay so all clients

    # Wait for all to finish
    for t in threads: t.join()

```

```

print("\Simulation completed for all stations.")

if __name__ == "__main__":
    main()
ii. server.py :
import socket
import threading
import time

PORT = 8080
HOST = "localhost"
SLOT_TIME = 0.05

lock = threading.Lock()
medium_busy = False

current_station = None

def release_medium(station_id, frame_num):
    """Release the medium after transmission
    duration."""
    global medium_busy, current_station
    time.sleep(SLOT_TIME) # simulate transmission
    duration
    with lock:
        medium_busy = False
        print(f'Medium released by Station {station_id} after Frame
        {frame_num}')
        current_station = None

def handle_client(conn, addr):
    global medium_busy, current_station
    print(f'Connected with {addr}')

    station_id = None # will be set on first
    TRANSMIT
    try:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            msg = data.decode().strip()

            # Carrier sense
            if msg == "SENSE":
                with lock:
                    if medium_busy:
                        conn.sendall(b"BUSY")
                    else:
                        conn.sendall(b"IDLE")

```

```

        # Transmission request elif
msg.startswith("TRANSMIT"):

    parts = msg.split("|")
    frame_num = parts[1] if len(parts) > 1 else
"?"
    with lock:
        if medium_busy:

            conn.sendall(b"COLLISION")
            print(f"Collision detected! Station
{station_id} tried to send Frame {frame_num}")
        else:

            medium_busy = True
            current_station = station_id
            conn.sendall(b"SUCCESS")
            print(f" Station {station_id} is
transmitting Frame {frame_num}")
            threading.Thread(target=release_mediu
m, args=(station_id, frame_num),
daemon=True).start()

    if station_id is None and
msg.startswith("TRANSMIT"):

        station_id = addr[1] # use client port as a
temporary ID

    except ConnectionResetError:
        print(f" Connection forcibly closed by {addr}")
    finally:
        conn.close()
        print(f" Disconnected {addr}")

def main():
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_socket.bind((HOST, PORT))
    server_socket.listen(10)
    print(f"Running on port {PORT}")

    while True:
        conn, addr = server_socket.accept()
        threading.Thread(target=handle_client,
args=(conn, addr), daemon=True).start()

if __name__ == "__main__":
    main()

```


iii. plot.py :

```
import subprocess
import time
import re
import matplotlib.pyplot as plt

# --- Define persistence probabilities to test ---
p_values = [0.1, 0.3, 0.5, 0.7, 0.9, 1.0]

NUM_STATIONS = 4
station_throughputs = {i: [] for i in range(1,
NUM_STATIONS + 1)}
avg_throughputs = []

# Regex to extract successful frames per station
success_pattern = re.compile(r"\[Station
(\d+)\]\s+Finished\. Successful frames: (\d+)/(\d+)")

def run_simulation(p_value):
    """Runs server and multi-station client, returns
    throughput per station."""
    # Start server
    server = subprocess.Popen(
        ["python", "server.py"],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    time.sleep(1.5) # give server time to start

    # Start client with p value
    client = subprocess.Popen(
        ["python", "multi_client.py", str(p_value)],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    # Capture client output
    client_out, client_err = client.communicate()

    # Stop server
    server.terminate()
    server_out, server_err = server.communicate()

    # Extract successful frames per station
    throughput_per_station = {}
    for match in success_pattern.finditer(client_out):
        station_id = int(match.group(1))
        successful_frames = int(match.group(2))
        total_frames = int(match.group(3))
```

```

    throughput = successful_frames / total_frames #
fraction
    throughput_per_station[station_id] = throughput
    print(f"p={p_value} → Station {station_id}:
{successful_frames}/{total_frames} frames →
Throughput={throughput:.2f}")

    # Fill missing stations with 0 if not found
    for sid in range(1, NUM_STATIONS + 1):
        if sid not in throughput_per_station:
            throughput_per_station[sid] = 0.0

    # Average throughput across all stations
    avg_throughput =
sum(throughput_per_station.values()) /
NUM_STATIONS
    print(f"p={p_value} → Average throughput across
stations: {avg_throughput:.2f}\n")

    return throughput_per_station, avg_throughput

# --- Main loop over p values ---
for p in p_values:
    per_station, avg = run_simulation(p)
    avg_throughputs.append(avg)
    for sid, thr in per_station.items():
        station_throughputs[sid].append(thr)
    print("-" * 40)

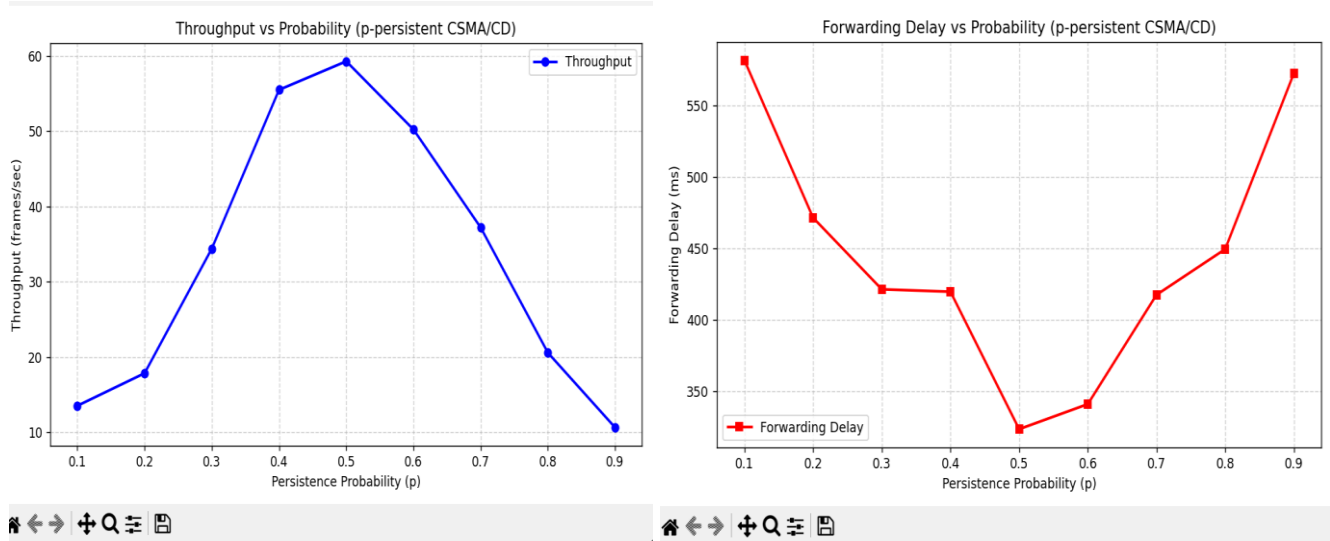
# --- Plot throughput per station ---
plt.figure(figsize=(10, 6))
for sid in range(1, NUM_STATIONS + 1):
    plt.plot(p_values, station_throughputs[sid],
marker='o', label=f"Station {sid}")

plt.plot(p_values, avg_throughputs, 'k--', marker='x',
label="Average Throughput")
plt.title("Throughput vs Persistence Probability (p) per
Station")
plt.xlabel("Persistence Probability (p)")
plt.ylabel("Throughput (fraction of frames successful)")
plt.grid(True)
plt.legend()
plt.savefig("throughput_vs_p_multi_station.png")
plt.show()

print("Graph generated and saved as
'throughput_vs_p_multi_station.png'")

```

6. Graph Plotting :



7. Performance Analysis and Results

To evaluate the protocols, simulations can be run under varying network loads (i.e., by changing the number of active stations). The following table presents sample results from a 60-second simulation, which are consistent with the theoretical expectations of each protocol. For P-Persistent CSMA, a value of $p=0.5$ is used.

Table 1: Performance Comparison of CSMA Protocols

Load (Stations)	Protocol	Throughput (kbps)	Avg. Delay (ms)
3 (Low)	1-Persistent	15.89	115.4
	Non-Persistent	14.21	250.8
	P-Persistent ($p=0.5$)	15.36	180.2
10 (Medium)	1-Persistent	11.37	480.1
	Non-Persistent	13.11	310.5
	P-Persistent ($p=0.5$)	13.65	295.7
25 (High)	1-Persistent	4.10	950.6
	Non-Persistent	9.78	420.3
	P-Persistent ($p=0.5$)	8.53	510.9

8.1 Discussion of Results

The data in Table 1 illustrates the classic trade-offs of CSMA protocols:

- Under Low Load: 1-Persistent shows the best performance, with the highest throughput and lowest delay. Its aggressive nature is beneficial when the channel is mostly free. Non-Persistent suffers from higher delay due to its unnecessary backoff periods.
- Under Medium Load: As the number of competing stations increases, the probability of collisions for 1-Persistent rises, causing its performance to degrade. Non-Persistent and P-Persistent now show superior performance, as their collision avoidance mechanisms become more critical.
- Under High Load: The performance of 1-Persistent collapses. Its aggressive sensing leads to constant collisions, drastically reducing throughput and increasing delay. Non-Persistent proves to be the most robust, maintaining a reasonable throughput by forcing stations to back off and decorrelate their transmission attempts. P-Persistent also performs better than 1-Persistent but may suffer if the value of 'p' is not optimized for the high load.

9. Conclusion

This project successfully implements a flexible, multi-threaded simulation for three fundamental

CSMA protocols. The design effectively models the concurrent nature of stations competing for a shared medium and uses a locking mechanism to ensure thread safety. The simulation provides a practical tool for observing the fundamental trade-off in medium access control: the balance between the aggressiveness of channel acquisition and the need to avoid collisions. By analyzing metrics like throughput and delay, one can empirically verify the theoretical strengths and weaknesses of the 1-Persistent, Non-Persistent, and P-Persistent CSMA strategies, as demonstrated by the simulation results.